

TOÁN RỜI RẠC VÀ THUẬT TOÁN
(DISCRETE MATHEMATIC AND ALGORITHMS)

Bài 4

Các phương pháp có sự thỏa hiệp
(Compromise Principle Based Methods)

Nguyễn Thị Hồng Minh

minhnhth@hus.edu.vn

Nội dung

- ❖ Phương pháp tham lam (Greedy)
- ❖ Phương pháp trực tuyến (Online)
- ❖ Phương pháp ngẫu nhiên (Randomized)
- ❖ Phương pháp xấp xỉ (Approximation)

Chú ý: Hầu hết các hình vẽ trong các bài giảng được sưu tầm từ internet và được trình bày theo quan điểm của giảng viên.

PHƯƠNG PHÁP THAM LAM

- ❖ Ý tưởng phương pháp
- ❖ Mô hình thuật toán
- ❖ Chứng minh tính đúng
- ❖ Đánh giá độ phức tạp
- ❖ Ví dụ

Ý tưởng phương pháp

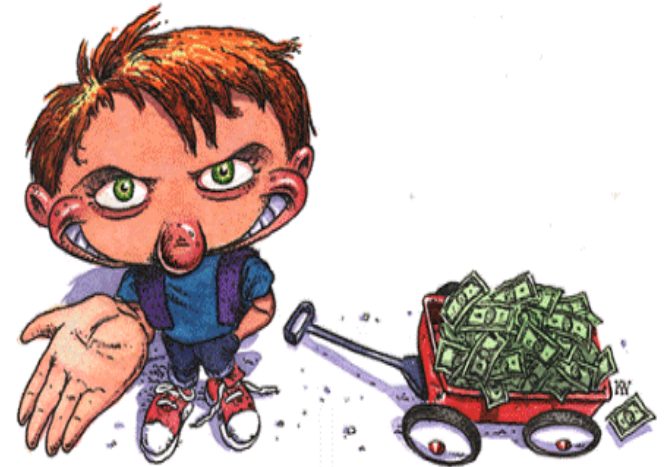
- Ý tưởng phương pháp tham lam

Xây dựng lời giải của bài toán với
việc chấp nhận những lựa chọn
“có vẻ” tốt nhất của từng giai đoạn.

Lựa chọn tối ưu cục bộ



Lời giải tối ưu toàn cục cho bài toán.



Ý tưởng phương pháp

- Ví dụ

Xếp số tiền 78 (nghìn đồng) với các tờ tiền mệnh giá 50, 10, 5 và 1 nghìn đồng sao cho số tờ tiền là ít nhất?

Áp dụng chiến lược tham lam

- Sắp các tờ tiền theo thứ tự giảm dần của mệnh giá
- Lấy tối đa số tờ mệnh giá lớn nhất có thể

$$\begin{aligned}\Rightarrow 78 &= 1 \text{ tờ } 50 + 2 \text{ tờ } 10 + 1 \text{ tờ } 5 + 3 \text{ tờ } 1 \\ &= 7 \text{ tờ}\end{aligned}$$

Ý tưởng phương pháp

- **Hai tính chất mang lại hiệu quả của chiến lược tham**
 - Tính chất của sự lựa chọn tham lam: một giải pháp tối ưu toàn cục có thể đạt được bằng cách thực hiện những chọn lựa tối ưu cục bộ.
 - Tính chất cấu trúc con tối ưu: một giải pháp tối ưu của bài toán chứa trong nó các giải pháp tối ưu của bài toán con.
- **Để đạt được những hiệu quả đó thì sự lựa chọn ở mỗi bước cần thỏa mãn:**
 - Khả thi (*Feasible*): Thỏa mãn các ràng buộc của bài toán
 - Tối ưu cục bộ (*Locally optimal*): Là lựa chọn tốt nhất trong các lựa chọn khả thi
 - Không thể thay đổi (*Irrevocable*): Một khi đã chọn, thì lựa chọn không thể thay đổi ở các bước tiếp theo

Lược đồ phương pháp

- Các thành phần cơ bản của một thuật toán tham:
 - Input: A - Tập hợp các đối tượng.
 - Output: S - Nghiệm tối ưu của bài toán.
 $S = \{x_1, x_2, \dots, x_n\}; x_i \in A; f(S)$ có giá trị tối ưu
 - Các hàm chức năng:
 - Hàm lựa chọn để xác định đối tượng tốt nhất tham gia vào lời giải ở mỗi giai đoạn – *BestSelect(A)*
 - Hàm xác định tính khả thi nếu đối tượng tốt nhất được lựa chọn có thể tham gia vào nghiệm của bài toán - *Acceptable(S, x)*.
 - Hàm đưa lựa chọn được chấp nhận vào nghiệm - *Integrate(S, x)*.
 - Hàm xác định một lời giải đã được xây dựng xong - *Is_solution(S)*.

Lược đồ phương pháp

- **Lược đồ**

```
Greedy (A)  $\equiv$  //Tìm lời giải tối ưu trên bộ dữ liệu A
|
|   S =  $\emptyset$  ;
|   while (A  $\neq \emptyset$  and !Is_solution(S))
|       |
|       |   x = BestSelect(A) ;
|       |   A = A \ {x};
|       |   if (Acceptable(S, x))
|       |       Integrate(S, x);
|       |
|       endw;
|   return S;
|
End.
```


Độ phức tạp giải thuật tham

- **Hai giai đoạn của thuật toán tham**

- Xử lý dữ liệu: Tạo tập đối tượng A

- Sắp xếp: $O(n \log n)$ $n=|A|$

- Tính toán, cắt tỉa: $O(n)$

- ...

- Sinh nghiệm tối ưu

- $O(n)$

- **Độ phức tạp (max):**

- Thường phụ thuộc vào giai đoạn xử lý dữ liệu

- Trong trường hợp sắp xếp: $O(n \log n)$

- So sánh với thuật toán sinh cấu hình nghiệm bằng phương pháp thử sai: $O(2^n)$

Chứng minh tính đúng thuật toán tham

- **Chiến lược tham luôn đưa ra kết quả tối ưu?**

- Câu trả lời: “Có thể là **không** đối với một số bài toán!”
- Ví dụ

Bài toán đổi tiền: Mệnh giá 10, 6, 1. Số tiền đổi là 55

Tham: $55 = 5 \times 10 + 5 \times 1 \Rightarrow 10$

Không tham: $55 = 4 \times 10 + 2 \times 6 + 3 \times 1 \Rightarrow 9$

$$55 = 3 \times 10 + 4 \times 6 + 1 \times 1 \Rightarrow 8$$

Chứng minh tính đúng thuật toán tham

- **Tính đúng của thuật toán tham**

- Tính đúng của lời giải: đảm bảo các ràng buộc/yêu cầu của bài toán.
- Tính tối ưu của lời giải: Lời giải tốt nhất trong các lời giải có thể.

- **Nhận định về tính đúng của thuật toán tham**

- Với cách tổ chức dữ liệu tốt, thuật toán tham sẽ đưa ra lời giải đúng và tối ưu
- Với dữ liệu đặc biệt (không điển hình) thuật toán không đưa ra lời giải tối ưu nhưng lời giải nhận được sẽ là “lời giải tốt”.
 - “Lời giải tốt”: sẽ có những lời giải có chất lượng kém hơn.
 - Tốt nhất = tối ưu

Chứng minh tính đúng thuật toán tham

- **Phương pháp chứng minh tính đúng của thuật toán tham**

- Sử dụng *Bất biến vòng lặp*: Tính đúng của lời giải được sinh ra.
- Sử dụng *Phản chứng*: Tính tối ưu của lời giải

Không dễ dàng tìm được phản chứng hoặc không có phản chứng!

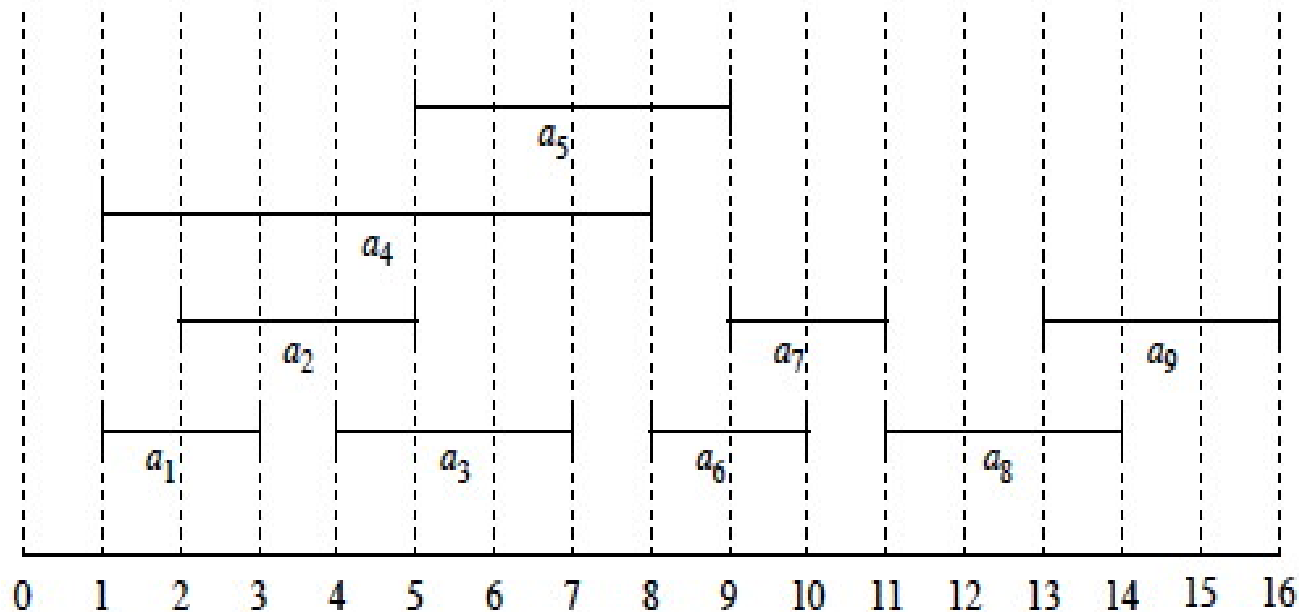
- Một số kỹ thuật khác
 - Staying ahead
 - Exchange arguments
 - Matroid
 - Greedoid

Xem thêm: Lecture notes “*Proof methods and greedy algorithms*”,
Magnus Lie Hetland, 5/2008

Bài toán lập lịch sử dụng tài nguyên

- **Activity-selection problem:** n hoạt động cùng sử dụng 1 tài nguyên. Mỗi hoạt động thứ i diễn ra trong thời gian s_i tới f_i ($s_i < f_i$) và toàn quyền sử dụng tài nguyên trong thời gian đã sắp xếp. Tìm dãy nhiều nhất các hoạt động có thể thực hiện được?
- **Ví dụ**

i	s_i	f_i
1	1	3
2	2	5
3	4	7
4	1	8
5	5	9
6	8	10
7	9	11
8	11	14
9	13	16



Bài toán lập lịch sử dụng tài nguyên

- **Phân tích:**

Input: $A = \{a_i \mid a_i = (a_i.s, a_i.f), a_i.s < a_i.f, i = 1..n\}$

Output: $S = \langle a'_1, a'_2, \dots, a'_m \rangle, a'_i \in A, a'_i.f < a'_{i+1}.s, i = 1..m-1; m = \max$

- **Chiến lược cho lựa chọn tham lam**

- Nếu chọn hoạt động kết thúc sớm nhất sẽ có cơ hội chọn được nhiều hoạt động phía sau.
- Sắp xếp dãy các hoạt động theo thứ tự tăng dần của thời gian kết thúc $(a_i.f < a_{i+1}.f, i = 1..n-1)$
- Tại mỗi bước xây dựng chuỗi hoạt động, chọn hoạt động có thể tiếp tục sau dãy hoạt động đã chọn và có thời gian kết thúc sớm nhất.
Ở bước i , đã có $a'_1, a'_2, \dots, a'_{i-1}$ cần chọn $a'_i \in A_i = A \setminus \{a'_1, a'_2, \dots, a'_{i-1}\}$ sao cho $a'_i.s > a'_{i-1}.f, a'_i.f = \min$ trong A_i
- Bắt đầu với a_1 .

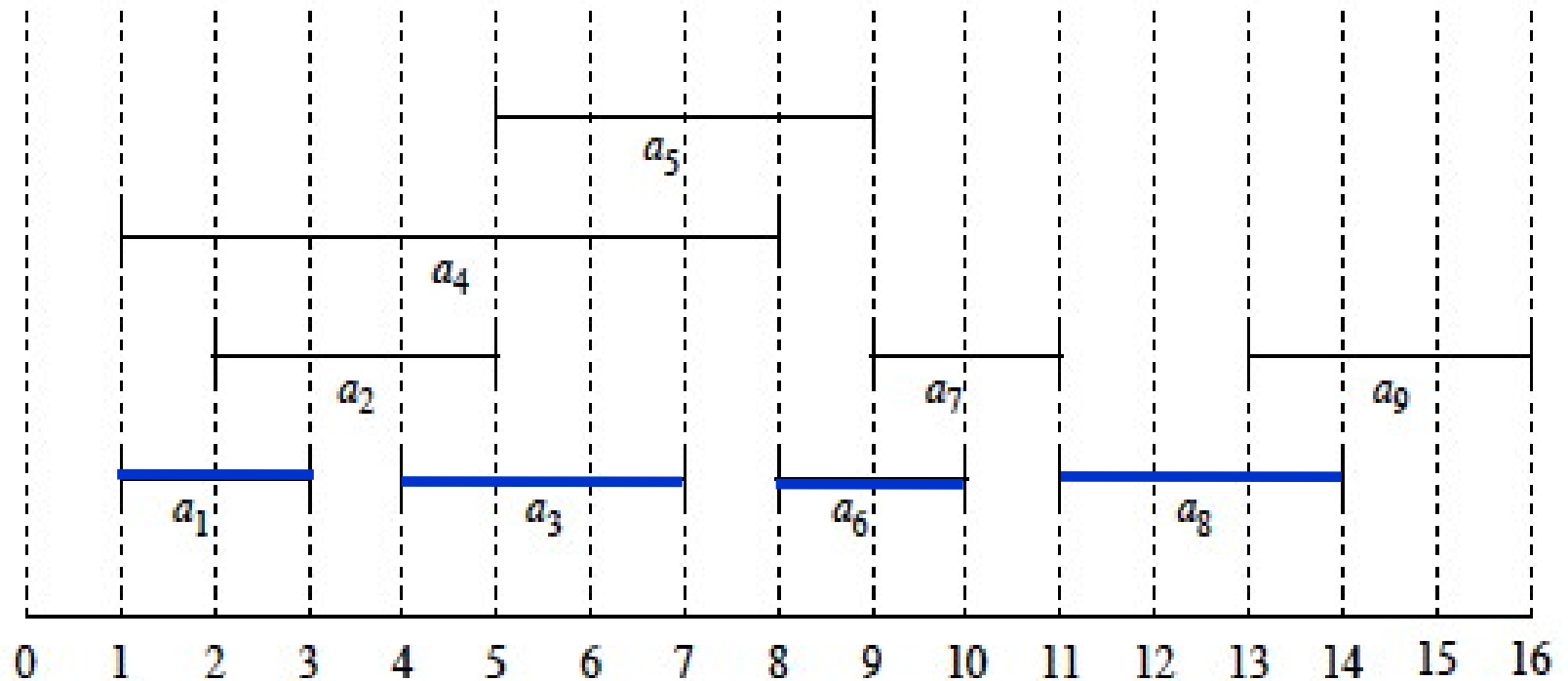
Bài toán lập lịch sử dụng tài nguyên

- Thuật toán:

```
Activity_Selector(A)  $\equiv$   
  Sort(A) ; //Sắp xếp dãy hoạt động  $A=\{a_1, a_2, \dots, a_n\}$  để  $a_i.f < a_{i+1}.f$  với mọi  $i$   
  S=<a1>; //Tập có thứ tự các hoạt động được chọn  
  i=1;  
  for (j=2..n)  
    if (aj.s ≥ ai.f)  
      S = S U <aj>; // U : Ghép aj vào sau dãy S đã chọn  
      i = j;  
    endif;  
  endfor;  
  return(S) ;  
End.
```

Bài toán lập lịch sử dụng tài nguyên

- **Minh họa**



Độ phức tạp giải thuật tham

- **Hai giai đoạn của thuật toán tham**

- Sắp xếp dãy các hoạt động:

`Sort (A)` - $O(n \log n)$

- Vòng lặp duyệt dãy các hoạt động để chọn hoạt động phù hợp

`for` ($i=2 \dots n$) - $O(n)$

- Độ phức tạp của thuật toán: $O(n \log n)$

Chứng minh tính đúng thuật toán tham

- **Tính đúng của lời giải**

- Định nghĩa “một dãy hoạt động triển vọng - *promising*”: Một dãy con các hoạt động $\langle a_{k_1}, a_{k_2}, \dots, a_{k_m} \rangle$ là triển vọng nếu thỏa mãn yêu cầu lựa chọn các hoạt động của bài toán đặt ra, đây là một đoạn con của lời giải cần tìm.
- Bất biến vòng lặp: $S = \langle a_{k_1}, a_{k_2}, \dots, a_{k_m} \rangle, a_{k_1} = a_1, k_m \leq i$ là dãy hoạt động triển vọng khi xét các hoạt động từ a_1 tới a_i .
- Các đặc trưng của bất biến vòng lặp:
 - Khởi tạo: $i = 1, S = \langle a_{k_1} \rangle = \langle a_1 \rangle$ là dãy triển vọng chỉ có một phần tử
 - Duy trì: Giả sử ở lần lặp thứ i chúng ta đã xét các hoạt động từ a_1 tới a_i và thu được dãy triển vọng S
Tại lần lặp $i+1$, xét hai trường hợp
 - a_{i+1} không được đưa vào S , theo giả thiết duy trì S là vẫn dãy triển vọng.
 - a_{i+1} được đưa vào S , ta được $S = \langle a_{k_1}, a_{k_2}, \dots, a_{k_m}, a_{k_{m+1}} \rangle, a_{k_1} = a_1, a_{k_{m+1}} = a_{i+1}$
Do tính chất đã được sắp của dãy A và điều kiện chấp nhận chọn a_{i+1} vào sau S ta có S' là dãy triển vọng.

Chứng minh tính đúng thuật toán tham

- **Tính đúng của lời giải**

- Các đặc trưng của bất biến vòng lặp:

- Duy trì: Giả sử ở lần lặp thứ i chúng ta đã xét các hoạt động từ a_1 tới a_i và thu được dãy triển vọng $S = \langle a_{k_1}, a_{k_2}, \dots, a_{k_m} \rangle, a_{k_1} = a_1, k_m \leq i$
Tại lần lặp thứ $i+1$ sẽ thu được một dãy triển vọng

- Kết thúc:

for ($j=2 \dots n$)

if ($a_j.s \geq a_i.f$)

$S = S \cup \langle a_i \rangle;$ // $\cup$: Ghép vào sau dãy S đã chọn

$i = j;$

Vòng lặp kết thúc sau $n-1$ lần cho ta dãy $S = \langle a_{k_1}, a_{k_2}, \dots, a_{k_m} \rangle, a_{k_1} = a_1, k_m \leq n$ là dãy triển vọng khi xét các hoạt động từ a_1 tới a_n , hay đây chính là một dãy hoạt động thỏa mãn điều kiện bài toán.

Tính đúng của lời giải được chứng minh.

Tính tối ưu của lời giải?

Chứng minh tính đúng thuật toán tham

- **Tính tối ưu của lời giải:** Lời giải tốt nhất trong các lời giải có thể.

- Sử dụng phản chứng:

Thuật toán tìm được $S = \langle a'_1, a'_2, \dots, a'_k \rangle, a'_1 = a_1$

Giả sử có dãy $S_1 = \langle a''_1, a''_2, \dots, a''_m \rangle, a''_1 = a_1$, S_1 tốt hơn S , tức là $m > k$.

Tìm sự vô lí?

- Nhận xét: $a'_i \cdot f \leq a''_i \cdot f, \forall i \leq k$ chứng minh bằng quy nạp

- $a'_1 = a_1 = a''_1$

- Tại bước $r-1$ đã có $a'_{r-1} \cdot f \leq a''_{r-1} \cdot f$ thì tại bước r do cách chọn a'_r của thuật toán ta có $a'_r \cdot f \leq a''_r \cdot f$ vì nếu $a'_r \cdot f > a''_r \cdot f$ thì a''_r đã được chọn vào vị trí a'_r

- Từ nhận xét trên ta thấy nếu $m > k$ tức là ta có $a'_k \cdot f \leq a''_k \cdot f$ và tồn tại a''_{k+1} mà $a'_k \cdot s < a'_k \cdot f \leq a''_k \cdot f \leq a''_{k+1} \cdot s < a''_{k+1} \cdot f$ tức là có thể chọn thêm a''_{k+1} vào sau S .

Vô lí vì thuật toán đã dừng khi tìm được a'_k

Một số bài toán giải bằng tham

- Optimal tape storage problem
- Continuous knapsack problem
- The union-find problem
- Shortest path problem: Dijkstra
- Min cost spanning trees: Prim, Kruskal

PHƯƠNG PHÁP TRỰC TUYẾN

❖ **Giới thiệu phương pháp**

❖ **Ví dụ**

- Bài toán “thuê hay mua”
- Bài toán “thư kí”
- Bài toán “thay trang và bộ đệm”

❖ **Một số bài toán và ứng dụng**

Giới thiệu phương pháp

❖ Thuật toán ngoại tuyến (offline algorithms)

- Dữ liệu input được nhập một lần toàn bộ trước khi thuật toán thực hiện.

❖ Thuật toán trực tuyến (online algorithms)

- Dữ liệu input không được nhập một lần toàn bộ trước khi thuật toán thực hiện: Do điều kiện về không gian nhớ hoặc tính chất dữ liệu.
- Tại mỗi bước thực hiện thuật toán phải có phương án kết quả mà không biết dữ liệu tiếp theo sẽ ra sao:
 - o Dự đoán quy luật phân bố dữ liệu.
 - o Xử lý trong trường hợp xấu nhất.

Giới thiệu phương pháp

❖ **Độ phức tạp và tính đúng của các thuật toán online**

- Độ phức tạp: tuyến tính theo kích thước dữ liệu vào do xử lý lần lượt, liên tiếp.
- Tính đúng:
 - o Thuật toán không cho ra kết quả chính xác duy nhất.
 - o Giảm thiểu các lỗi chiến lược (minimizes strategic errors)
- Quan tâm tới đánh giá hiệu năng (performance evaluation) thuật toán: Kết quả chấp nhận tốt nhất trong điều kiện (thời gian, không gian) cho phép.

Bài toán “thuê hay mua”

❖ Bài toán “Thuê hay mua” (Buy vs Rent problem)

- Ví dụ cụ thể: Giá mua giày trượt tuyết là 500\$. Giá thuê giày là 50\$/lần. Thuê hay mua?
- Câu trả lời:
 - o Phụ thuộc vào số lần sẽ chơi
 - o Nhưng input này chưa biết trước!
 - o Tại mỗi lần chơi:
 - Nếu lần trước đã mua giày thì thuật toán coi như kết thúc.
 - Nếu chưa, cần quyết định sẽ thuê hay mua dựa vào dữ kiện là số lần chơi.

Bài toán “thuê hay mua”

❖ Phân tích

- Bài toán tối ưu, input được lần lượt đưa vào ở mỗi lần thực hiện thuật toán - giải quyết bằng online algorithm.
 - o Input: Giá thuê, giá mua
 - o Output: t - thời điểm mua đạt hiệu quả cao nhất
- Quan điểm xử lý: phân tích “cạnh tranh” (competitive analysis)
 - o Xác định tỉ lệ cạnh tranh (competitive ratio - CR):

$$CR = \frac{\text{Online Algorithm Solution}}{\text{Optimal Solution}}$$

- o Tìm ra thời điểm t , CR đạt giá trị tối ưu nhất

Bài toán “thuê hay mua”

❖ Lời giải

- $t=1, CR=500/50 = 10; \quad t=2, CR=550/100 = 5.5$

- Tổng quát

- o Khi $t \leq 10 \quad CR = \frac{500 + 50(t-1)}{50t} = 1 + \frac{9}{t}$

- o Khi $t \geq 10 \quad CR = \frac{500 + 50(t-1)}{500} = 0.9 + \frac{t}{10}$

- o Tỷ lệ cạnh tranh tối ưu $CR=1.9$ khi $t=10$.

Mua giày ở lần chơi thứ 10!

- **Tính thỏa hiệp:** luật dự báo (predicting perspective) “nếu đã chơi 10 lần thì khả năng rất cao là sẽ có 10 lần chơi tiếp theo”.

Bài toán “thư kí”

❖ Bài toán “thư kí” (Secretary Problem)

- Phỏng vấn n người cho một vị trí. Giữa hai người có thể so sánh được sau khi phỏng vấn cả hai. Mỗi lần phỏng vấn 1 người và đưa ra quyết định ngay tuyển hay không. Làm sao để tuyển được người tốt nhất?

Tên gọi hay hơn: bài toán tuyển dụng?

- Ý tưởng thuật toán:
 - o Pha thử: Phỏng vấn n/e người được lấy ngẫu nhiên từ n người, không tuyển ai.
 - o Pha tuyển: Phỏng vấn lần lượt những người tiếp theo và chấp nhận người đầu tiên tốt hơn tất cả những người đã phỏng vấn.
- Có thể chứng minh thuật toán theo ý tưởng trên cho khả năng chọn được người tốt nhất với xác suất $1/e$ (37%)

Bài toán “thư kí”

❖ Thuật toán tìm người tốt nhất với xác suất $1/e$

- n ứng viên được phân hạng theo thứ tự $1, 2, \dots, n$

Dãy các ứng viên được phỏng vấn là một hoán vị π của n giá trị $1, 2, \dots, n$

Pha thử: tìm giá trị $m = \min(\pi[1], \dots, \pi[t])$

Pha tuyển: tìm giá trị j đầu tiên mà $j > t$ và $\pi[j] < m$

- Xác suất để $\pi[j] = 1$ là

$$\sum_{j=t+1}^n P(\pi[j] = 1 \text{ and pick the best person at } j\text{-th date})$$
$$= \sum_{j=t+1}^n P(\pi[j] = 1 \& \min(\pi[1], \dots, \pi[j-1]) \in \{\pi[1], \dots, \pi[t]\}) = \sum_{j=t+1}^n \frac{1}{n} \cdot \frac{t}{j-1}$$

Bài toán “thư kí”

❖ Thuật toán tìm người tốt nhất với xác suất $1/e$

- Xác suất để $\pi[j]=1$ là

$$p = \sum_{j=t+1}^n \frac{1}{n} \cdot \frac{t}{j-1} = \frac{t}{n} \left(\sum_{j=1}^{n-1} \frac{1}{j} - \sum_{j=1}^{t-1} \frac{1}{j} \right) \approx \frac{t}{n} \cdot (\ln n - \ln t) = \frac{t}{n} \cdot \ln \frac{n}{t}$$

- p tốt nhất là $1/e$ với giá trị $t=n/e$
tl. Xác suất để chọn người tốt nhất ở vị trí j ($\pi[j]=1$) là $1/e \approx 37\%$
- Tuy nhiên thuật toán có thể thất bại khi:
 - o $\pi[j]=1$ với $j \leq t$ tiêu chuẩn đã bị đặt quá cao.
 - o $\exists k$ mà $\pi[k] > \pi[j]$ và $\pi[k] < \min(\pi[1], \dots, \pi[t])$ gặp ứng viên “hơi tốt hơn” trước

Giải pháp?

Bài toán “thay trang và bộ đệm”

❖ Bài toán “thay trang và bộ đệm” (Paging and Caching)

- Mô hình tổ chức bộ nhớ phân cấp: Memory - Cache - Hard disk
Dung lượng lớn, tốc độ chậm (thấp) - Dung lượng nhỏ, tốc độ nhanh (cao)
- Bài toán:
 - o Trong quá trình xử lý, dữ liệu (page) chuyển từ bộ nhớ level thấp sang level cao (*cache*)
 - o Khi cần xử lý dữ liệu, tìm trong *cache*, nếu có thì xử lý (*have a hit*), nếu không (*have a miss*) thì chuyển dữ liệu vào.
 - o Khi thực hiện một phép chuyển dữ liệu nếu *cache* đã đầy sẽ thực hiện xóa bớt để thay thế.
 - o Vấn đề đặt ra: Sẽ xóa dữ liệu nào?
 - Quyết định online
 - Mục tiêu làm giảm các *miss* trong tương lai

Bài toán “thay trang và bộ đệm”

❖ Thuật toán

- Thuật toán trực tuyến:
 - o Input : dãy các yêu cầu truy xuất dữ liệu (*page*).
 - o Output: Quyết định khi có một *miss* xảy ra và *cache* đã đầy thì sẽ xóa *page* nào để thay thế sao cho giảm thiểu các *miss* có thể xảy ra trong tương lai.
- Một số chiến lược thay thế trang
 - o FIFO (First-In First-Out)
 - o LIFO (Last-In First-Out)
 - o CLOCK (Clock Replacement)
 - o LRU (Least Recently Used)
 - o LFU (Least Frequently Used)

Một số bài toán online và ứng dụng

❖ Một số bài toán online

- Self-Organizing Data Structure: Tự tổ chức dữ liệu trong cấu trúc dữ liệu đảm bảo truy xuất hiệu quả.
- The k-Server Problem: Truy cập dữ liệu từ hệ thống mạng qua các server sao cho thời gian được giảm thiểu (lớp bài toán bộ đệm và thay trang)
- Metrical Task System: Mô hình hóa các bài toán online

❖ Lĩnh vực ứng dụng

- Large Network: Network Switches, Transmission Control Protocol (TCP)
- Competitive Auctions: Đấu giá cạnh tranh
- Financial, Trade, Social ???

Bài tập 3

1. Thiết kế và phân tích thuật toán

- Chọn một bài toán có thể giải bằng thuật toán tham. Thiết kế và phân tích giải thuật giải bài toán đó.

(Nên kết hợp để làm phong phú thêm bài tập 2)

2. Tìm hiểu chiến lược giải thuật: Chọn 1 trong 2

- Trình bày về matroid, greedoid trong chứng minh tính đúng giải thuật tham lam.
- Trình bày về giải thuật online cho một bài toán: phát biểu bài toán, tính thỏa hiệp, tính online, lời giải, đánh giá, ứng dụng.

Nộp bài: Các file liên quan nén trong tệp tên **BT3.<hoten>.<hoten>.rar**

gửi về minhnhth@gmail.com trong vòng hai tuần sai khi kết thúc bài

NP- Complete

- **Một số tranh luận về P, NP, NPC**
 - Chưa tìm được một thuật toán đa thức cho bất kì bài toán NPC nào (mặc dù có khá nhiều bài toán NPC)
 - Chưa có chứng minh nào chỉ ra rằng không tồn tại thuật toán đa thức cho bất kì bài toán NPC (mặc dù đã rất cố gắng).
 - Hầu hết các nhà khoa học về lí thuyết tính toán đều ***tin rằng*** NPC là intractable (tức là khó và $P \neq NP$).
 - Thực tế???

NP- Complete

- **Một số kĩ thuật đối phó với bài toán NPC**

- *Xấp xỉ*: Thay vì tìm nghiệm tối ưu, mà tìm lời giải gần nhất với sự tối ưu.
- *Ngẫu nhiên*: Sử dụng phép ngẫu nhiên để làm thuật toán chạy nhanh hơn, chấp nhận một số trường hợp thất bại với xác suất nhỏ.
- *Hạn chế*: Hạn chế cấu trúc của dữ liệu đầu vào để thuật toán chạy nhanh hơn.
- *Tham số hóa*: Thuật toán có thể chạy nhanh hơn nếu một số tham số nào đó của dữ liệu đầu vào được cố định (are fixed).
- *Heuristic*: Thuật toán có thể thực hiện tốt chấp nhận được trong nhiều trường hợp theo kinh nghiệm nào đó. Nhưng có thể không chứng minh được cả hai tính chất chạy nhanh và cho ra kết quả chính xác.

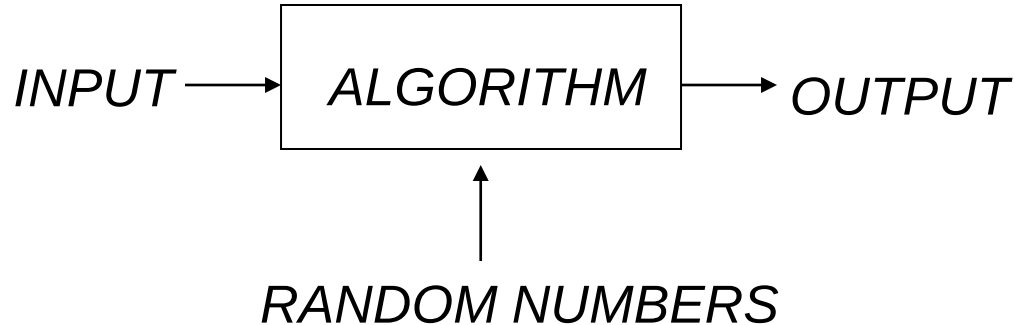
PHƯƠNG PHÁP NGẪU NHIÊN

- ❖ Ý tưởng phương pháp
- ❖ Thuật toán Monte Carlo
- ❖ Thuật toán Las Vegas
- ❖ Bài toán cặp điểm gần nhất

Ý tưởng phương pháp

❖ Thuật toán ngẫu nhiên (randomized algorithm):

- Đưa yếu tố ngẫu nhiên vào mỗi bước thực hiện thuật toán



Thuật toán có thể chứa các lệnh dạng:

```
x ← random()  
if (x = 0)  
    do A ...  
else {x = 1}  
    do B ...
```

hoặc

```
x ← random()  
do A with x
```

Ý tưởng phương pháp

❖ Thuật toán ngẫu nhiên (randomized algorithm):

- 2 dạng của thuật toán ngẫu nhiên:
 - Với bài toán tối ưu: đưa ra một lời giải tốt nhất với thời gian trung bình tốt nhất.
 - Với bài toán quyết định: lời giải có thể mắc sai sót nhưng với xác suất nhỏ phụ thuộc vào thời gian thực hiện.
- Tính thỏa hiệp: Chấp nhận kết quả với mục tiêu hiệu quả thời gian thực hiện thuật toán.

Thuật toán Monte Carlo

❖ Thuật toán ngẫu nhiên Monte Carlo:

- Thuật toán áp dụng đối với các bài toán quyết định
- Luôn cho ra lời giải của bài toán, nhưng kết quả có thể có sai sót.
- Xác suất cho ra lời giải đúng tỉ lệ với thời gian thực hiện
- Ví dụ áp dụng: QuickSort ngẫu nhiên, kiểm tra số nguyên tố, nhân ma trận...

Thuật toán Las Vegas

❖ Thuật toán ngẫu nhiên Las Vegas

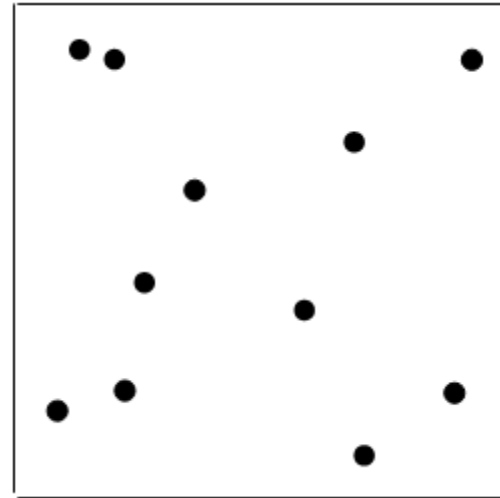
- Thuật toán áp dụng đối với các bài toán tối ưu
- Lời giải nhận được là tốt nhất
- Thời gian thực hiện trung bình của thuật toán có giới hạn (tốt)
- Ví dụ áp dụng: Phân tích số nguyên ra thừa số nguyên tố, căn bậc 2 modulo p , bài toán tám hậu...

Bài toán cặp điểm gần nhất

❖ Tìm kiếm cặp điểm gần nhất (closest-pair problem)

- Bài toán: Cho một tập n điểm trong mặt phẳng, tìm cặp điểm gần nhau nhất (theo nghĩa khoảng cách Euclid)

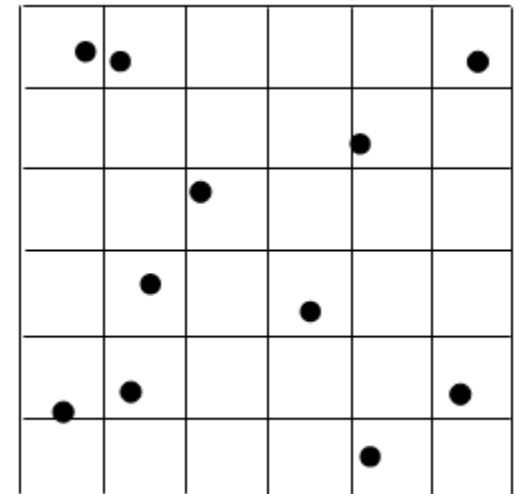
- Các phương pháp truyền thống
 - Vét cạn (Brute-force): $O(n^2)$
 - Chia để trị: $O(n \log n)$



Bài toán cặp điểm gần nhất

❖ Phương pháp ngẫu nhiên giải bài toán

- Ý tưởng:
 - Chia các điểm thành một số miền khác nhau
 - Tính khoảng cách các điểm trong mỗi miền để tìm khoảng cách nhỏ nhất
 - Gần tương tự như chia để trị, nhưng không thực hiện **tổng hợp**
- Vấn đề: Chia miền
 - Lưới?
 - Kích thước?
 - Có thể đưa yếu tố ngẫu nhiên?



Bài toán cặp điểm gần nhất

❖ Thuật toán ngẫu nhiên

Input: Tập S bao gồm n điểm $x_1, x_2, \dots, x_n, S \subseteq R^2$.

Output: $d = \min\{d(x_i, x_j)\}; i, j = 1..n$.

Step 1: Chọn ngẫu nhiên tập $S_1 = \{x_{i_1}, x_{i_2}, \dots, x_{i_m}\}$ với $m = n^{2/3}$. Tìm khoảng cách nhỏ nhất trong S_1 , kí hiệu là δ .

Step 2: Thiết lập lưới vuông T với kích thước δ .

Step 3: Thiết lập 4 lưới vuông T_1, T_2, T_3, T_4 từ T với kích thước gấp đôi 2δ .

Step 4: Với mỗi lưới T_i , tìm phân hoạch $S = S_1^{(i)} \cup S_2^{(i)} \cup \dots \cup S_k^{(i)}, i = 1..4$ với mỗi $S_k^{(i)}$ là giao khác rỗng của S với các ô lưới của T_i .

Step 5: Với mỗi cặp $(x_p, x_q) \in S_k^{(i)}$, tính $d(x_p, x_q)$. Trả về kết quả là khoảng cách bé nhất tìm được.

Bài toán cặp điểm gần nhất

❖ Thuật toán ngẫu nhiên

Input: Tập S bao gồm n điểm $x_1, x_2, \dots, x_n, S \subseteq R^2$.

Output: $d = \min\{d(x_i, x_j)\}; i, j = 1..n$.

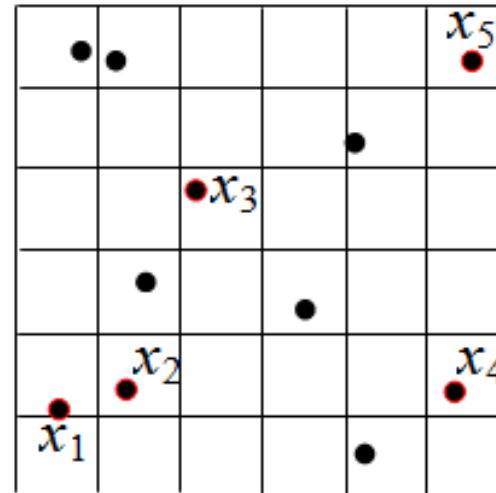
Step 1: Chọn ngẫu nhiên tập $S_1 = \{x_{i_1}, x_{i_2}, \dots, x_{i_m}\}$ với $m = n^{2/3}$. Tìm δ .

Step 2: Lưới vuông T với kích thước δ .

$$n = 11; m = 5$$

$$S_1 = \{x_1, x_2, \dots, x_5\}$$

$$\delta = d(x_1, x_2)$$

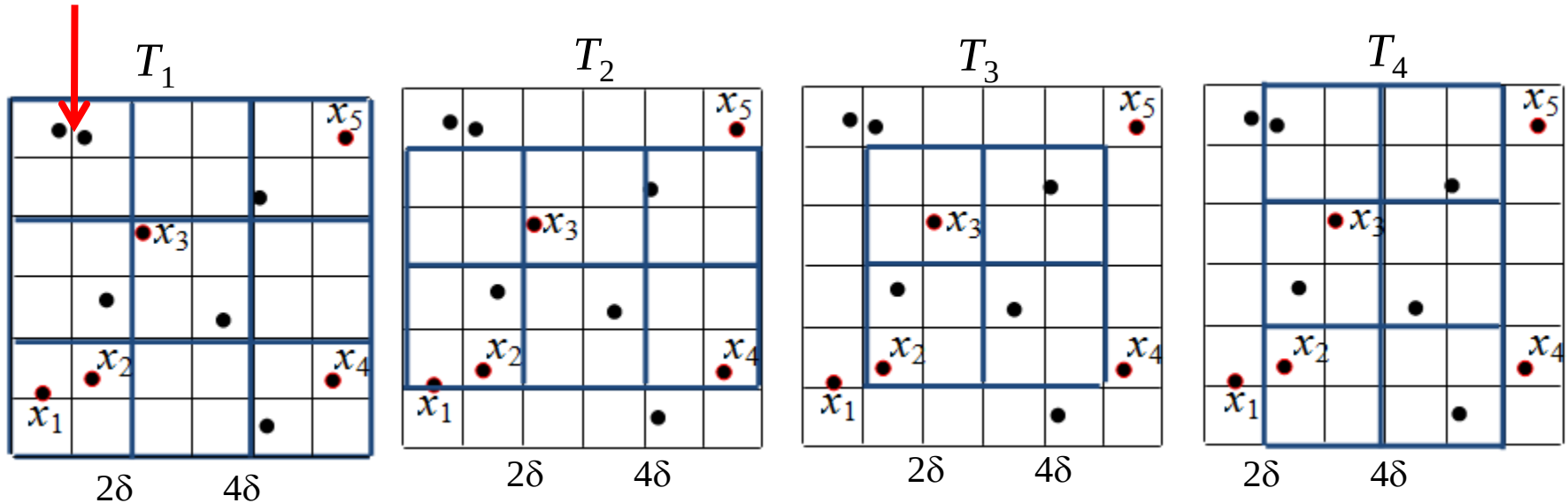


δ

Bài toán cặp điểm gần nhất

❖ Thuật toán ngẫu nhiên

Step 3..5: Lưới vuông T_1, T_2, T_3, T_4 với kích thước 2δ . Tính $d(x_p, x_q)$ trong mỗi $S_k^{(i)}$.
Trả về kết quả là khoảng cách bé nhất tìm được.



Bài toán cặp điểm gần nhất

❖ Đánh giá thuật toán

Step 1:

Lấy ngẫu nhiên $n^{2/3}$ điểm $O(n)$

Tìm khoảng cách nhỏ nhất của $n^{2/3}$ điểm $O(n^{2/3} \log n^{2/3})$

Tổng cộng $O(n)$

Step 2..4: $O(n)$

Step 5: Số các khoảng cách được tính toán trong phân hoạch T

$O(n)$ với xác suất $1 - 2e^{-cn^{1/6}}$?

PHƯƠNG PHÁP XẤP XỈ

- ❖ **Khái niệm**
- ❖ **Mục tiêu của phương pháp**
- ❖ **Đánh giá thuật toán**
- ❖ **Bài toán áp dụng**

Khái niệm

- Lớp bài toán quyết định khó trong NP là NP -đầy đủ (NPC).
- Dạng tối ưu của các bài toán này là N-khó (NP -hard). Hiện chưa có thuật toán thời gian đa thức để giải các bài toán này.
- Sẽ làm gì để giải những bài toán như vậy?
- Nhắc lại: thuật toán thử sai dạng nhánh cận cũng không đảm bảo giải quyết được những bài toán như vậy.
- Các tiếp cận khác: giải xấp xỉ, hi vọng thời gian sẽ nhanh hơn.

Khái niệm

- Tại sao cần thuật toán xấp xỉ:
 - Đôi khi một lời giải đủ tốt là đủ.
 - Trong thực tế có thể dữ liệu vào không thật chính xác.
- Thuật toán xấp xỉ dựa trên một số cảm nhận, kinh nghiệm (heuristic) nào đó của bài toán.
- Một heuristic là một qui tắc được rút ra từ kinh nghiệm hơn là một khẳng định được chứng minh bằng toán học.

Khái niệm

- Tại sao cần thuật toán xấp xỉ:
 - Đôi khi một lời giải đủ tốt là đủ.
 - Trong thực tế có thể dữ liệu vào không thật chính xác.
- Thuật toán xấp xỉ dựa trên một số cảm nhận, kinh nghiệm (heuristic) nào đó của bài toán.
- Một heuristic là một qui tắc được rút ra từ kinh nghiệm hơn là một khẳng định được chứng minh bằng toán học.

Mục tiêu của thuật toán

- **Mục tiêu:**

- Đảm bảo thời gian chạy thuật toán là đa thức.
- Đảm bảo tìm được nghiệm đủ tốt (gần với nghiệm tối ưu).

- **Khó khăn:**

- Cần chứng minh nghiệm tìm được đủ tốt, tức là gần với nghiệm tối ưu nhưng lại không biết nghiệm tối ưu!

Đánh giá thuật toán

- **Hệ số chính xác**

- Nếu sử dụng giải thuật xấp xỉ, chúng ta có thể muốn biết độ chính xác của giải thuật đó?
- Sai số quan hệ (Relative error): càng nhỏ càng tốt $\rightarrow 0$):

$$re(s_a) = \frac{f(s_a) - f(s^*)}{f(s^*)}$$

trong đó: s^* là một nghiệm chính xác

s_a là nghiệm xấp xỉ.

- Hệ số chính xác (accurate ratio): càng lớn càng tốt $\rightarrow 1$

$$re(s_a) = \frac{f(s_a)}{f(s^*)}$$

Bài toán áp dụng

- Thiết kế thuật toán xấp xỉ cho một số bài toán
 - Thuật toán phụ thuộc vào bài toán được giải, mỗi bài toán có cách xây dựng thuật toán khác nhau
 - Ví dụ:
 - Approximation Algorithms for TSP
 - Approximation Algorithms for Knapsack Problem
 - Approximation Algorithms for Vertex Cover Problem
 - ...